

Sistema Web para Análise Comparativa do Poder de Compra entre Países

Caio de Luccas Rosolen^{1*}; Mauricio Acconcia Dias²

¹Bacharel em Administração. Rua Antonio Malaquias Paes, 313 – Jardim Guanabara; CEP 13190-656 Monte Mor, São Paulo, Brasil

²Doutor em Ciências da Computação e Matemática Computacional. Av. Doutor Maximiliano Baruto – Jardim Universitário; CEP 13607-339 Araras, São Paulo, Brasil

*autor correspondente: caiodeluccas@gmail.com

Sistema Web para Análise Comparativa do Poder de Compra entre Países

Resumo

Este trabalho teve como objetivo desenvolver um sistema web para análise comparativa do poder de compra entre países, utilizando preços locais de produtos e indicadores salariais como base para cálculo de métricas de acessibilidade. A pesquisa foi conduzida por meio de um estudo de caso aplicado, com foco na concepção e implementação de uma solução de software composta por módulos de coleta, persistência, processamento, API e interface web. Para isso, utilizaram-se dados de preços obtidos via Canopy API e indicadores salariais obtidos na ILOSTAT, armazenados em banco de dados PostgreSQL e processados sob demanda pelo backend. Os resultados demonstraram a viabilidade técnica da arquitetura modular adotada, evidenciando a integração entre os componentes do sistema, a preservação histórica dos dados coletados, a exposição padronizada das informações por meio de API REST e a exibição dos resultados em interface web. Observou-se que a solução implementada foi capaz de atender ao escopo do produto mínimo viável, permitindo consultas comparativas por país e por produto. Concluiu-se que o sistema atingiu o objetivo proposto e estabeleceu uma base consistente para futuras expansões, incluindo ampliação de indicadores, análises históricas e evolução da camada de apresentação.

Palavras-chave: acessibilidade de compra; integração de dados; arquitetura modular; API REST; produto mínimo viável.

Web-Based System for Comparative Analysis of Purchasing Power Across Countries

Abstract

This study aimed to develop a web system for comparative analysis of purchasing power across countries, using local product prices and wage indicators as the basis for calculating affordability metrics. The research was conducted as an applied case study, focusing on the design and implementation of a software solution composed of data collection, persistence, processing, API, and web interface modules. To achieve this, price data obtained through the Canopy API and wage indicators obtained from ILOSTAT were used, stored in a PostgreSQL database, and processed on demand by the backend. The results demonstrated the technical feasibility of the adopted modular architecture, highlighting the integration among system components, the historical preservation of collected data, the standardized exposure of information through a REST API, and the presentation of results through a web interface. It was observed that the implemented solution was able to meet the scope of the minimum viable product, allowing comparative queries by country and by product. It was concluded that the system achieved its proposed objective and established a consistent basis for future expansions, including the incorporation of additional indicators, historical analyses, and further development of the presentation layer.

Keywords: affordability; data integration; modular architecture; REST API; minimum viable product.

Introdução

A crescente disponibilidade de dados públicos e de interfaces de programação de aplicações (APIs) tem ampliado as possibilidades de desenvolvimento de sistemas capazes de integrar, processar e disponibilizar informações econômicas de forma automatizada. Contudo, do ponto de vista da Engenharia de Software, a simples coleta de dados não basta; esse cenário demanda arquiteturas modulares e escaláveis, capazes de garantir a rastreabilidade e a evolução do sistema frente a fontes de dados heterogêneas (Pressman e

Maxim, 2016; Sommerville, 2016). Essa necessidade técnica torna-se ainda mais crítica quando o objetivo é realizar comparações internacionais, campo no qual análises baseadas exclusivamente em conversão cambial nominal frequentemente falham em representar o esforço real de aquisição de bens.

Para superar essa limitação da conversão cambial, instituições como o World Bank utilizam o International Comparison Program para produzir paridades de poder de compra que controlam as diferenças nos níveis de preços entre economias (World Bank, 2026). Embora tais medidas operem em escala macroeconômica, elas fundamentam a premissa deste trabalho: a comparação internacional de valores exige métricas que observem a acessibilidade econômica em contextos locais distintos. Sob essa ótica, a relação entre preço, salário e tempo de trabalho necessário para a compra de um item constitui uma abordagem prática e intuitiva para medir o poder de compra, conforme demonstrado por Ashenfelter (2001).

Entretanto, a aplicação dessas teorias econômicas em ferramentas digitais dinâmicas ainda apresenta lacunas. O presente trabalho não busca substituir indicadores macroeconômicos consolidados, mas sim desenvolver uma solução de software que integre preços locais e indicadores salariais da International Labour Organization (ILO, 2026) em uma métrica aplicada de acessibilidade. A priorização de dados da ILO justifica-se pela organização de bases comparáveis de salários e jornadas, elementos essenciais para compor o cálculo do "esforço de compra" de forma automatizada em um Produto Mínimo Viável (MVP).

Dessa forma, a oportunidade reside na criação de soluções computacionais voltadas à integração desses dados em nível operacional. Um sistema web capaz de coletar preços em diferentes países e relacioná-los historicamente com indicadores econômicos oferece não apenas uma consulta comparativa simplificada, mas também a base para análises temporais sobre a evolução do poder de compra.

Diante desse cenário, o objetivo deste trabalho foi desenvolver um sistema web para análise comparativa do poder de compra entre países. O foco recai na arquitetura de software, na integração automatizada de dados e na disponibilização de métricas de acessibilidade de compra por meio de uma aplicação web funcional.

Metodologia

A pesquisa foi conduzida por meio de um estudo de caso aplicado, tendo como objeto de análise o desenvolvimento de um sistema web destinado à coleta, armazenamento, processamento e disponibilização de dados econômicos obtidos a partir de APIs públicas. O trabalho concentrou-se na concepção e implementação de uma solução de software capaz de integrar informações de preços de produtos e indicadores salariais de diferentes países, permitindo consultas comparativas por meio de uma interface padronizada.

Os dados utilizados no sistema consistiram, principalmente, em preços de produtos comercializados e em indicadores de salários-mínimo, médio e média de horas trabalhadas, empregados como base para o cálculo de métricas de acessibilidade de compra. Esses dados foram obtidos por meio de fontes públicas e serviços externos integrados ao sistema, sendo armazenados de forma estruturada em banco de dados relacional para posterior processamento e consulta.

Sob a perspectiva da Engenharia de Software, o estudo contemplou as etapas de levantamento de requisitos, definição da arquitetura, modelagem de dados, implementação dos serviços de coleta e processamento, desenvolvimento da API e construção da interface

web do sistema. A solução foi organizada em módulos independentes, executados em contêineres, com o objetivo de favorecer a manutenção, a portabilidade e a evolução futura da aplicação.

Tipo e abordagem da pesquisa

O trabalho caracterizou-se como uma pesquisa aplicada, de abordagem qualitativa e quantitativa, desenvolvida por meio de estudo de caso. Foi aplicada porque buscou resolver um problema a prático por meio da construção de uma solução funcional de software. A abordagem qualitativa esteve presente na definição da arquitetura, na modelagem dos componentes e na organização das decisões de implementação. A abordagem quantitativa manifestou-se na coleta, tratamento e exposição de dados numéricos utilizados pelo sistema para geração das comparações entre países e produtos.

Os dados empregados foram predominantemente secundários, uma vez que foram obtidos a partir de APIs públicas e fontes institucionais externas. A utilização dessas fontes contribuiu para a padronização da coleta, para a reprodutibilidade dos procedimentos e para a viabilidade técnica do produto mínimo viável desenvolvido no escopo do trabalho.

Fontes de dados e critérios de seleção

Os dados utilizados no sistema foram obtidos a partir de fontes externas integradas à solução desenvolvida. Para os preços dos produtos, foi utilizada a Canopy API, empregada como fonte de consulta no escopo do produto mínimo viável (MVP). Para os indicadores econômicos e salariais, foi utilizada a base International Labour Organization Statistics (ILOSTAT), mantida pela Organização Internacional do Trabalho, selecionada por disponibilizar dados padronizados e comparáveis entre países.

A seleção dos indicadores salariais foi orientada pela necessidade de compor métricas de acessibilidade capazes de relacionar preços locais ao esforço necessário para aquisição de produtos. Nesse sentido, foram priorizados indicadores de remuneração mensal e por hora, salário-mínimo legal e jornada de trabalho, uma vez que a própria ILOSTAT organiza sua base de estatísticas de salários e tempo de trabalho a partir de medidas como remuneração mensal, remuneração por hora, salários mínimos estatutários e horas efetivamente trabalhadas. A utilização desse conjunto de variáveis mostrou-se adequada ao objetivo do MVP, por permitir a construção de comparações aplicadas entre preços e renda do trabalho em diferentes países (International Labour Organization [ILO], 2026).

A escolha dessas fontes foi orientada por critérios de disponibilidade técnica, padronização dos dados e viabilidade de integração com a arquitetura proposta. Considerando o escopo aplicado do trabalho e a necessidade de construção de uma solução funcional em tempo compatível com o desenvolvimento do TCC, optou-se por utilizar APIs públicas ou acessíveis por integração automatizada, capazes de fornecer dados estruturados para armazenamento e processamento no sistema.

Os países incluídos no MVP foram definidos a partir da disponibilidade simultânea das fontes utilizadas e da presença da Amazon como referência para obtenção padronizada dos preços. O Brasil foi mantido pela relevância contextual do estudo e por sua aderência ao público-alvo do trabalho, enquanto os Estados Unidos foram incluídos em razão de sua centralidade econômica e da ampla disponibilidade de dados na fonte de preços adotada. Os demais países foram escolhidos entre localidades com presença da Amazon, buscando

compor uma amostra internacional reduzida, porém suficiente para validação da solução proposta.

A definição dos produtos também foi realizada de forma intencional, considerando a viabilidade técnica e econômica da coleta automatizada. Em razão das limitações de consumo e custo da API utilizada para obtenção dos preços, o conjunto inicial foi restringido a um número reduzido de itens, distribuídos em diferentes categorias de consumo. Essa decisão permitiu manter o sistema dentro dos limites operacionais do MVP, sem comprometer a demonstração das funcionalidades centrais da arquitetura implementada.

Arquitetura geral do sistema

A arquitetura do sistema foi definida com base em uma abordagem modular, inspirada em princípios de microsserviços, com o objetivo de favorecer a manutenção, a escalabilidade e a evolução independente dos componentes. A solução foi implementada por meio de contêineres Docker, nos quais cada módulo do sistema foi isolado em seu próprio ambiente de execução.

A estrutura da aplicação foi organizada em componentes responsáveis pela coleta automatizada de dados, processamento das informações, persistência em banco de dados relacional e disponibilização dos resultados por meio de uma API REST, além de um módulo destinado à camada de apresentação. A orquestração e a comunicação entre esses componentes foram realizadas com o uso do Docker Compose, permitindo a execução integrada dos serviços de forma padronizada e reproduzível.

No fluxo adotado, o módulo de coleta realiza a obtenção de dados em APIs externas, o módulo de processamento executa a agregação e o cálculo das métricas derivadas, o banco de dados PostgreSQL mantém o armazenamento estruturado e histórico das informações, e a API fornece acesso padronizado aos dados processados. Por fim, o frontend atua como camada de apresentação, permitindo ao usuário consultar os resultados gerados pelo sistema.

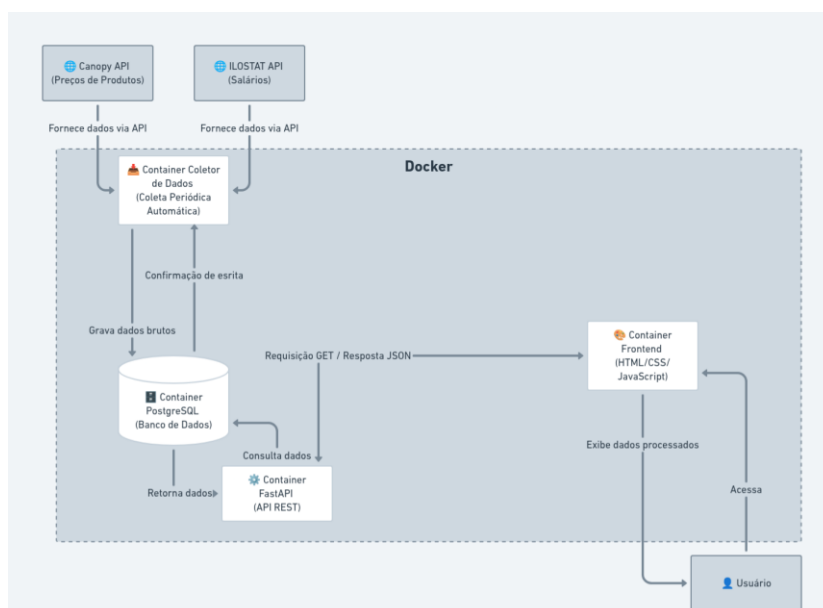


Figura 1. Diagrama de arquitetura geral do sistema em nível de containers (C4 Model)
Fonte: Dados originais da pesquisa

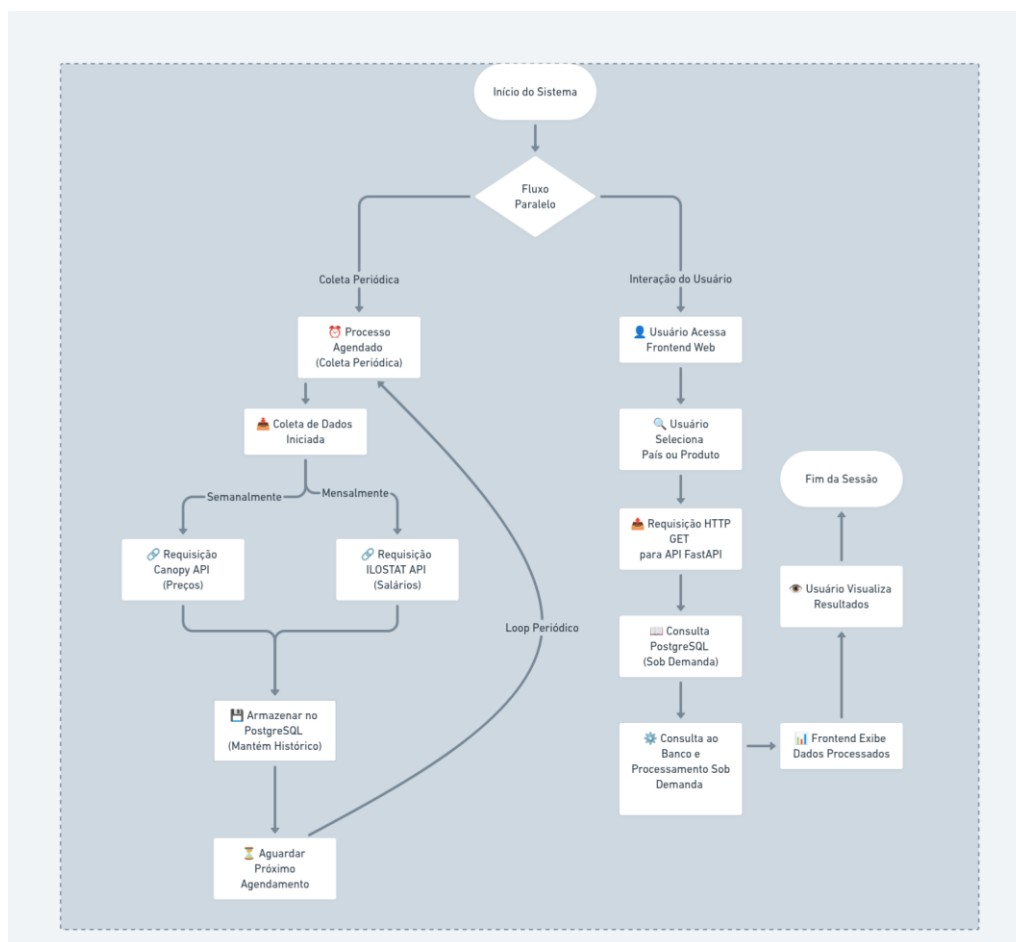


Figura 2. Fluxo de interação do usuário e consulta sob demanda
Fonte: Dados originais da pesquisa

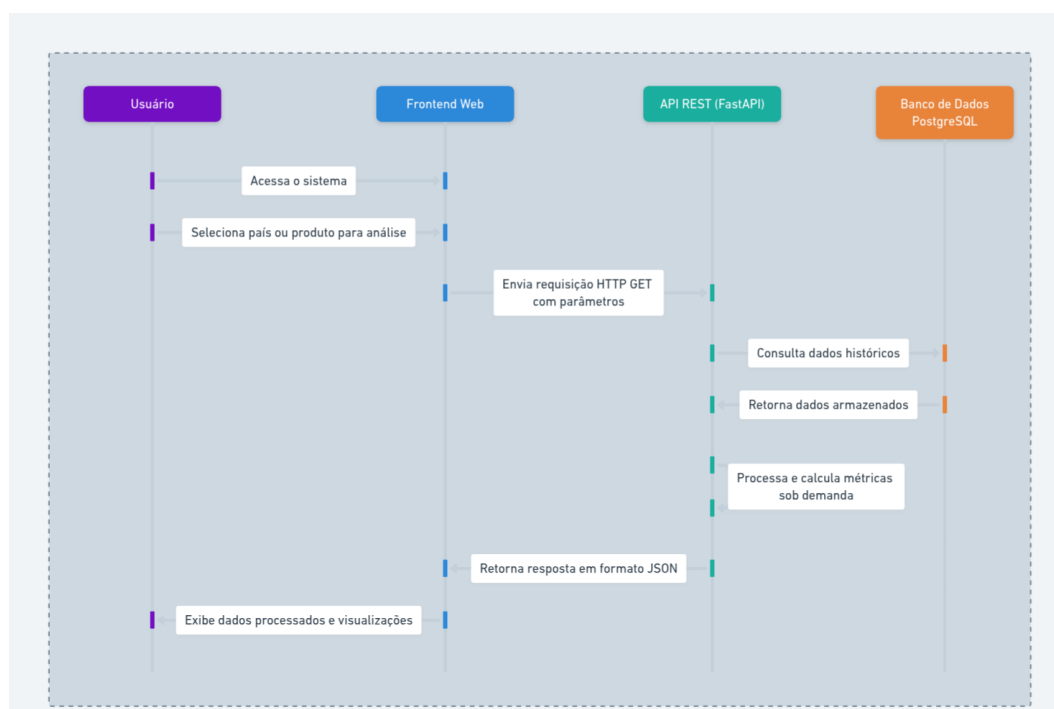


Figura 3. Diagrama de sequência da consulta sob demanda
Fonte: Dados originais da pesquisa

Tecnologias utilizadas no desenvolvimento do sistema

A implementação do sistema utilizou a linguagem de programação Python, escolhida em razão de sua ampla adoção em aplicações de backend, automação e integração com APIs. Para a persistência dos dados, foi adotado o sistema gerenciador de banco de dados PostgreSQL, em função de sua robustez, conformidade com o padrão SQL e suporte à integridade referencial.

A camada de exposição dos dados foi implementada com o framework FastAPI, que possibilitou a criação de endpoints REST de forma eficiente, além de fornecer documentação automática das APIs por meio do padrão OpenAPI. O processamento das informações foi realizado no backend, permitindo a geração sob demanda das métricas comparativas a partir dos dados armazenados.

Na camada de apresentação, foi desenvolvido um frontend web com React, utilizando Vite como ferramenta de build e organização do projeto, além de Tailwind CSS para estilização da interface. Essa combinação foi adotada com o objetivo de construir uma aplicação leve, modular e adequada ao escopo do produto mínimo viável, permitindo a exibição dos dados processados de forma simplificada ao usuário final.

A solução foi empacotada e executada por meio de contêineres Docker, permitindo o isolamento dos diferentes componentes do sistema, a padronização do ambiente de execução e a facilitação do processo de implantação e manutenção da aplicação. A orquestração dos serviços foi realizada com Docker Compose, viabilizando a comunicação entre banco de dados, coleta, processamento, API e frontend de maneira integrada.

Levantamento de Requisitos

O levantamento de requisitos foi realizado com foco na definição das funcionalidades essenciais para a validação da proposta do sistema, considerando o escopo de um produto mínimo viável (MVP). Os requisitos foram definidos a partir dos objetivos do trabalho, priorizando a integração entre os componentes da solução, a persistência histórica dos dados, a disponibilidade das informações processadas e a possibilidade de evolução futura do sistema.

Os requisitos funcionais correspondem às ações e comportamentos que o sistema deve executar para atender às necessidades definidas no trabalho.

RF01 – Coleta automatizada de preços: o sistema deve realizar a coleta automática e periódica de preços de produtos em diferentes países por meio da integração com a Canopy API.

RF02 – Coleta de indicadores salariais: o sistema deve consumir dados da API da ILOSTAT para obtenção de indicadores salariais utilizados nas análises comparativas.

RF03 – Persistência histórica dos dados: o sistema deve armazenar os dados coletados de forma cronológica, preservando registros anteriores e permitindo a formação de base histórica para análises futuras.

RF04 – Consulta por país e por produto: o sistema deve permitir consultas por país, retornando os produtos disponíveis em uma localidade, e por produto, retornando a comparação do mesmo item entre diferentes países.

RF05 – Cálculo de métricas de acessibilidade: o sistema deve calcular, sob demanda, métricas derivadas a partir da relação entre preço local e indicadores salariais, incluindo percentual do salário comprometido e horas necessárias para aquisição do produto.

RF06 – Exposição dos dados por API: o sistema deve disponibilizar os resultados processados por meio de uma API REST, em formato estruturado, para consumo pela camada de apresentação.

RF07 – Interface web para visualização: o sistema deve disponibilizar uma interface web para consulta dos dados processados pelo usuário final.

Os requisitos não funcionais correspondem aos critérios técnicos e de qualidade que orientam a implementação e a operação do sistema.

RNF01 – Arquitetura modular baseada em contêineres: o sistema deve ser organizado em módulos independentes, executados em contêineres Docker, de forma a favorecer isolamento, portabilidade e manutenção.

RNF02 – Persistência em banco de dados relacional: o sistema deve utilizar o PostgreSQL para armazenamento estruturado dos dados, garantindo integridade referencial e consistência das informações.

RNF03 – Processamento sob demanda: as métricas comparativas devem ser calculadas no backend no momento da requisição, evitando a necessidade de armazenamento redundante de resultados processados.

RNF04 – Padronização da comunicação: a troca de informações entre os componentes do sistema deve ocorrer por meio de requisições HTTP e respostas em formato JSON.

RNF05 – Execução local integrada: o sistema deve ser capaz de ser executado integralmente em ambiente local, facilitando testes, validação e demonstrações do MVP.

RNF06 – Compatibilidade entre processamento e serialização: o backend deve realizar o tratamento dos tipos de dados gerados no processamento para garantir compatibilidade com a serialização das respostas consumidas pelo frontend.

Modelagem e persistência dos dados

O banco de dados do sistema foi modelado segundo uma abordagem relacional, contemplando tabelas de referência, tabelas de dados históricos e tabelas auxiliares destinadas aos processos de coleta, validação e processamento das informações.

Entre as estruturas implementadas, destacam-se a tabela `countries`, responsável pelo cadastro dos países analisados, e a tabela `country_translations`, utilizada para suportar a tradução dos nomes dos países. As tabelas `products` e `categories` foram empregadas para a organização lógica dos produtos e de suas categorias, enquanto a tabela `product_asins` foi utilizada para o mapeamento dos códigos de busca por país. O histórico de preços coletados é armazenado na tabela `price_history`.

Para os indicadores econômicos, foram utilizadas as tabelas `labor_indicators`, responsável pelo catálogo dos indicadores disponíveis, e `labor_indicators_history`, destinada ao armazenamento histórico dos valores coletados por país e ano de referência. Complementarmente, a tabela `sources` registra as fontes de dados utilizadas, e a tabela `staging_labor_indicators` atua como estrutura de apoio para carga e validação dos dados antes de sua persistência definitiva.

A modelagem adotada buscou assegurar a integridade referencial, a rastreabilidade histórica das coletas e a separação entre dados cadastrais, dados históricos e estruturas auxiliares de processamento. A descrição detalhada do banco de dados, incluindo campos, tipos, chaves e restrições, encontra-se apresentada no Apêndice B – Dicionário de dados do banco de dados.

Cargas iniciais e dados não coletados automaticamente

No sistema, algumas tabelas exigem dados estáticos de configuração para que os processos de coleta, processamento e análise possam ser executados corretamente. Esses dados são inseridos por meio de scripts de carga inicial (*seed*), não sendo obtidos por coleta dinâmica, mas definidos previamente como parte da configuração do domínio da aplicação.

Entre os registros de carga inicial, incluem-se as fontes externas utilizadas pelo sistema, cadastradas na tabela *sources*, com destaque para a ILOSTAT API e a Canopy API. Também foi utilizada a tabela *labor_indicators*, responsável pelo catálogo dos indicadores empregados nas requisições à API da ILOSTAT. No escopo do MVP, foram selecionados indicadores relacionados a salário médio mensal, salário médio por hora, salário-mínimo mensal e média de horas semanais trabalhadas, por serem suficientes para o cálculo das métricas comparativas adotadas no sistema.

As tabelas *products* e *categories* também foram preenchidas por carga inicial, contemplando os produtos analisados no estudo. No escopo do MVP, foram cadastrados itens de diferentes categorias de consumo, incluindo bebidas, vestuário, brinquedos e eletrônicos. Essa definição prévia foi necessária para restringir o universo de coleta a um conjunto viável de produtos, considerando as limitações operacionais e o custo de consumo da API utilizada para obtenção dos preços.

A tabela *product_asins*, por sua vez, foi utilizada para mapear os códigos de busca empregados na identificação dos produtos por país, permitindo maior consistência na coleta automatizada em diferentes localidades.

Tabelas de dados coletados (dinâmicas)

As tabelas de dados coletados são utilizadas para armazenar registros históricos, sem sobrescrita ou exclusão dos dados anteriormente obtidos. Essa abordagem permite a construção de séries temporais e a análise da evolução dos valores ao longo do tempo.

A tabela *price_history* armazena os registros brutos de preços dos produtos coletados periodicamente. A coleta de preços foi configurada com frequência semanal, permitindo acompanhar variações ao longo do tempo e viabilizando análises comparativas a partir dos dados mais recentes disponíveis.

A tabela *labor_indicators_history*, por sua vez, armazena os registros históricos dos indicadores econômicos e salariais por país e ano de referência. Embora esses indicadores sejam atualizados oficialmente em periodicidade distinta da coleta de preços, optou-se por realizar coletas mensais com o objetivo de incorporar eventuais atualizações disponíveis na fonte externa e manter a base de dados continuamente atualizada.

A diferença entre as periodicidades adotadas foi definida em função da natureza de cada conjunto de dados. Enquanto os preços dos produtos podem variar com maior frequência, os indicadores salariais tendem a sofrer alterações menos recorrentes, o que justificou a coleta mensal dessa segunda categoria.

A adoção dessa estrutura histórica permite preservar os dados brutos em sua forma original e oferece suporte a futuras análises temporais, inclusive comparações da evolução conjunta entre preços e indicadores salariais.

Implementação da API

A API foi estruturada de forma modular, separando a definição das rotas da lógica de processamento e acesso aos dados. Essa organização contribuiu para a manutenção do código, para a clareza da arquitetura e para a evolução independente das funcionalidades expostas pelo sistema.

Para o tratamento dos dados históricos coletados, utilizou-se a biblioteca Pandas, empregada na manipulação das consultas retornadas pelo banco de dados e na aplicação das regras de cálculo das métricas comparativas. Essa escolha técnica permitiu realizar agregações e transformações de forma eficiente, viabilizando o processamento dos dados em memória antes da geração das respostas finais.

A API disponibiliza endpoints REST responsáveis por atender consultas em diferentes níveis de granularidade, incluindo consultas por país e por produto. As respostas são retornadas em formato JSON, garantindo interoperabilidade com o frontend e com eventuais integrações futuras.

Processamento sob demanda

Em vez de armazenar previamente todos os resultados processados, a solução foi concebida com processamento sob demanda. Nesse modelo, os cálculos são executados no momento da requisição, a partir dos dados históricos persistidos no banco de dados.

Quando uma consulta é realizada, o sistema recupera os dados necessários, aplica as regras de transformação e calcula, em tempo real, as métricas comparativas baseadas na relação entre preços e indicadores salariais. Essa estratégia reduz a complexidade da persistência, preserva a integridade dos dados brutos e garante que a análise apresentada ao usuário seja gerada com base nos registros mais recentes disponíveis.

Aspectos técnicos de implementação

Um desafio técnico relevante na implementação do sistema esteve relacionado à diferença entre os tipos de dados utilizados no processamento analítico, especialmente estruturas do NumPy e do Pandas, e os formatos compatíveis com a serialização em JSON. Para tratar essa incompatibilidade, foi implementada uma camada de conversão responsável por transformar tipos numéricos específicos em tipos nativos de Python e por tratar valores nulos, assegurando a integridade das respostas retornadas pela API e a compatibilidade com a camada de frontend.

Além disso, a utilização de contêineres Docker contribuiu para o isolamento dos componentes da aplicação e de suas respectivas dependências. Essa estratégia favoreceu a padronização do ambiente de execução, a portabilidade da solução e a reprodutibilidade da arquitetura implementada, reduzindo problemas decorrentes de diferenças entre ambientes de desenvolvimento e implantação.

Interface web do sistema

A camada de apresentação do sistema foi implementada por meio de uma interface web destinada à consulta dos dados processados pelo backend. O frontend foi desenvolvido com foco no escopo do produto mínimo viável, priorizando simplicidade de navegação, clareza na exibição das métricas comparativas e separação entre lógica de apresentação e processamento dos dados.

A interface foi estruturada para consumir os endpoints disponibilizados pela API REST e apresentar ao usuário consultas por país e por produto, permitindo a visualização das métricas calculadas a partir dos preços coletados e dos indicadores salariais. O desenvolvimento dessa camada teve como objetivo oferecer uma forma padronizada e acessível de interação com o sistema, sem transferir ao cliente regras de negócio ou processamento pesado.

No escopo do MVP, o frontend contemplou páginas de navegação inicial, consulta por país, consulta por produto e páginas textuais de contextualização do sistema, mantendo a responsabilidade de processamento concentrada no backend.

Resultados e Discussão

Esta seção apresenta os resultados obtidos com a implementação do sistema no escopo do produto mínimo viável (MVP), contemplando a execução da coleta automatizada, o armazenamento histórico dos dados, o processamento sob demanda das métricas comparativas e a disponibilização das informações por meio da API e da interface web. Os resultados apresentados têm como finalidade demonstrar a viabilidade técnica da solução proposta e discutir, sob a perspectiva da Engenharia de Software, o desempenho da arquitetura adotada e sua aderência aos objetivos definidos no trabalho.

As evidências analisadas incluem registros de execução dos serviços, respostas obtidas a partir das rotas implementadas, dados persistidos no banco relacional e resultados exibidos na interface do sistema. Em conjunto, esses elementos permitem verificar o funcionamento integrado da solução e discutir o comportamento dos principais componentes desenvolvidos, desde a coleta dos dados externos até a apresentação final das informações ao usuário.

O código-fonte do sistema, incluindo scripts de coleta, processamento, definição do banco de dados e configuração dos contêineres, foi mantido em repositório público para fins de versionamento, rastreabilidade e reprodutibilidade técnica.¹ Os registros completos de execução utilizados como evidência do funcionamento da aplicação encontram-se apresentados no Apêndice A.

Execução do sistema e coleta de dados

A execução do sistema demonstrou a capacidade de realizar a coleta automatizada de dados econômicos a partir das APIs externas configuradas. O processo de coleta ocorre de forma periódica e independente da interação do usuário, sendo responsável por obter dados brutos de preços de produtos e indicadores salariais.

A Figura 4 apresenta um exemplo de execução do processo de coleta, evidenciando a comunicação com as fontes externas e a inserção dos dados no banco de dados.

¹ Repositório do projeto: github.com/caiodeluccas/tcc-comparador-precos-MBA-USP

```
caio@caio-System-Product-Name:~/Documents/MBA/comparacao_precos arquivos/comparacao_precos$ docker logs coletor_dados_app
2026-02-10 18:06:26 [INFO] __main__: Iniciando Microserviço Coletor
2026-02-10 18:06:26 [INFO] __main__: Conexão com Banco de Dados: OK
2026-02-10 18:06:26 [INFO] __main__: --- INICIANDO EXECUÇÃO ÚNICA DE COLETA ---
2026-02-10 18:06:26 [INFO] __main__: Coletando indicador salarial: EAR_4MTH_SEX_CUR_NB_A
2026-02-10 18:06:26 [INFO] job_salary: Iniciando coleta ILO para o indicador: EAR_4MTH_SEX_CUR_NB_A...
2026-02-10 18:06:30 [INFO] db_connector: Tabela staging_salary limpa com sucesso (Truncate).
2026-02-10 18:06:30 [INFO] job_salary: Sucesso: Dados de 1 migrados para a tabela final.
2026-02-10 18:06:30 [INFO] __main__: Coletando indicador salarial: MWG_2MTH_SEX_CUR_NB_A
2026-02-10 18:06:30 [INFO] job_salary: Iniciando coleta ILO para o indicador: MWG_2MTH_SEX_CUR_NB_A...
2026-02-10 18:06:32 [WARNING] job_salary: Nenhum dado retornado para MWG_2MTH_SEX_CUR_NB_A
2026-02-10 18:06:32 [INFO] __main__: Coletando indicador salarial: QLS_HW_AVE_NB_A
2026-02-10 18:06:32 [INFO] job_salary: Iniciando coleta ILO para o indicador: QLS_HW_AVE_NB_A...
2026-02-10 18:06:33 [WARNING] job_salary: Nenhum dado retornado para QLS_HW_AVE_NB_A
2026-02-10 18:06:33 [INFO] __main__: --- INICIANDO COLETA DE PREÇOS (CANOPY/AMAZON) ---
2026-02-10 18:06:33 [INFO] job_products: Iniciando coleta Canopy via Mapeamento Global...
2026-02-10 18:06:39 [INFO] job_products: SUCESSO: COCA_ZERO_12P em BR -> 46.68
2026-02-10 18:06:47 [INFO] job_products: SUCESSO: COCA_ZERO_12P em US -> 8.39
2026-02-10 18:06:55 [INFO] job_products: SUCESSO: COCA_ZERO_12P em ES -> 9.96
2026-02-10 18:07:02 [INFO] job_products: SUCESSO: HAVAIANAS_TOP em BR -> 34.5
2026-02-10 18:07:13 [INFO] job_products: SUCESSO: HAVAIANAS_TOP em US -> 14.33
2026-02-10 18:07:20 [INFO] job_products: SUCESSO: HAVAIANAS_TOP em ES -> 24
2026-02-10 18:07:20 [INFO] __main__: --- TAREFA FINALIZADA COM SUCESSO ---
caio@caio-System-Product-Name:~/Documents/MBA/comparacao_precos arquivos/comparacao_precos$
```

Figura 4. Registros de log da execução do coletor de dados

Fonte: Resultados originais da pesquisa

Persistência e organização dos dados

Os dados coletados pelo sistema são armazenados em banco de dados PostgreSQL, preservando o histórico das coletas realizadas e evitando a sobrescrita dos registros anteriormente obtidos. Essa abordagem permite manter a rastreabilidade das informações e cria uma base adequada para análises comparativas e temporais, tanto no escopo atual do MVP quanto em evoluções futuras da solução.

No contexto da implementação desenvolvida, a persistência foi organizada de forma a separar dados cadastrais, dados históricos e estruturas auxiliares de processamento. A observação dos registros persistidos indicou consistência entre os dados coletados nas fontes externas e os valores armazenados na base relacional.

A organização adotada mostrou-se adequada ao objetivo do sistema, uma vez que permitiu preservar os dados em sua forma bruta e, ao mesmo tempo, oferecer suporte ao processamento sob demanda realizado pela API. Além disso, a manutenção do histórico constitui um dos principais ativos da solução proposta, pois viabiliza não apenas a consulta dos dados atuais, mas também futuras análises sobre a evolução conjunta entre preços, salários e métricas de acessibilidade.

As Figuras 5, 6 e 7 apresentam exemplos de registros persistidos na base de dados, ilustrando a organização das informações históricas utilizadas pelo sistema.

```
^Caio@caio-System-Product-Name:~/Documents/MBA/comparacao_precos arquivos/comparacao_precos$ docker exec -it postgres_db psql -U pc_app_admin -d price_comp -c "\dt"
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | categories | table | pc_app_admin
public | countries | table | pc_app_admin
public | country_stats | table | pc_app_admin
public | country_translations | table | pc_app_admin
public | exchange_rates | table | pc_app_admin
public | price_history | table | pc_app_admin
public | product_asins | table | pc_app_admin
public | products | table | pc_app_admin
public | salary_history | table | pc_app_admin
public | salary_indicators | table | pc_app_admin
public | sources | table | pc_app_admin
public | staging_salary | table | pc_app_admin
(12 rows)
```

```
caio@caio-System-Product-Name:~/Documents/MBA/comparacao_precos arquivos/comparacao_precos$
```

Figura 5. Listagem das tabelas criadas no banco de dados PostgreSQL

Fonte: Resultados originais da pesquisa

```
caio@caio-System-Product-Name:~/Documents/MBA/comparacao_precos arquivos/comparacao_precos$ docker exec -it postgres_db psql -U pc_app_admin -d price_comp -c "SELECT * FROM salary_hist
ory LIMIT 10;"
```

id_salary	id_country	id_indicator	id_source	salary_value	currency	reference_year	collection_date
24	233	1	1	5985.29	LCU	2024	2026-01-29 12:00:02.373339+00
28	63	1	1	525.09	LCU	2024	2026-01-29 12:00:02.373339+00
71	190	1	1	890.49	LCU	2023	2026-01-29 12:00:02.373339+00
76	190	1	1	1045.72	LCU	2023	2026-01-29 12:00:02.373339+00
79	249	1	1	3239.98	LCU	2023	2026-01-29 12:00:02.373339+00
99	82	1	1	1797.20	LCU	2022	2026-01-29 12:00:02.373339+00
133	164	1	1	45206.17	LCU	2019	2026-01-29 12:00:02.373339+00
169	144	1	1	30416.00	LCU	2014	2026-01-29 12:00:02.373339+00
179	14	1	1	3013.00	LCU	2010	2026-01-29 12:00:02.373339+00
121	3	1	1	13202.25	LCU	2020	2026-01-29 12:00:02.373339+00

(10 rows)

Figura 6. Registros de indicadores salariais coletados e armazenados

Fonte: Resultados originais da pesquisa

```
caio@caio-System-Product-Name:~/Documents/MBA/comparacao_precos arquivos/comparacao_precos$ docker exec -it postgres_db psql -U pc_app_admin -d price_comp -c "SELECT sku, id_country, p
rice, currency, collection_timestamp FROM price_history ORDER BY collection_timestamp DESC LIMIT 15;"
```

sku	id_country	price	currency	collection_timestamp
HAVAIANAS_TOP	3	24.00	EUR	2026-02-10 18:06:33.511983+00
COCA_ZERO_12P	1	46.68	BRL	2026-02-10 18:06:33.511983+00
COCA_ZERO_12P	2	8.39	USD	2026-02-10 18:06:33.511983+00
COCA_ZERO_12P	3	9.96	EUR	2026-02-10 18:06:33.511983+00
HAVAIANAS_TOP	1	34.50	BRL	2026-02-10 18:06:33.511983+00
HAVAIANAS_TOP	2	14.33	USD	2026-02-10 18:06:33.511983+00
COCA_ZERO_12P	1	46.68	BRL	2026-01-29 12:00:03.752248+00
COCA_ZERO_12P	2	8.39	USD	2026-01-29 12:00:03.752248+00
COCA_ZERO_12P	3	9.96	EUR	2026-01-29 12:00:03.752248+00
HAVAIANAS_TOP	1	28.89	BRL	2026-01-29 12:00:03.752248+00
HAVAIANAS_TOP	2	15.95	USD	2026-01-29 12:00:03.752248+00
HAVAIANAS_TOP	3	21.26	EUR	2026-01-29 12:00:03.752248+00

(12 rows)

Figura 7. Registros históricos de preços coletados por país

Fonte: Resultados originais da pesquisa

Exposição e consumo dos dados via API

A API desenvolvida mostrou-se capaz de expor os dados persistidos no banco de forma padronizada, por meio de endpoints REST voltados às consultas por país e por produto. As respostas são retornadas em formato JSON, contendo os dados já processados e organizados de acordo com as regras definidas na camada de backend. Esse comportamento confirmou a adequação da API como elemento intermediário entre a persistência relacional e a camada de apresentação do sistema.

Durante os testes realizados, observou-se que a API foi capaz de recuperar os dados históricos armazenados, aplicar as regras de transformação necessárias e disponibilizar ao frontend respostas estruturadas para consumo imediato. Dessa forma, a camada de apresentação pôde operar sem necessidade de processamento adicional relevante, mantendo no backend a responsabilidade pelas regras de negócio e pelos cálculos das métricas comparativas.

Esse resultado reforça uma das decisões centrais da arquitetura adotada no trabalho: concentrar a lógica de processamento na API e utilizar o frontend como camada de visualização e interação. Tal abordagem contribui para a separação de responsabilidades entre os componentes, reduz a complexidade da interface web e favorece a manutenção e a evolução futura da solução.

As Figuras 8, 9 e 10 apresentam exemplos de respostas retornadas pela API em consultas realizadas pelo frontend, evidenciando a estrutura dos dados processados e a forma como essas informações são disponibilizadas para a camada de apresentação.

```
2026-02-10 17:59:49 [INFO] processor.data_processor: Conexão com o banco de dados estabelecida pelo DataEngine.
2026-02-10 17:59:49 [INFO] processor.data_processor: EconomicProcessor inicializado e pronto para processamento.
2026-02-10 17:59:49 [INFO] API_ECONOMICA: Motor de Processamento (EconomicProcessor) inicializado com sucesso.
2026-02-10 18:06:14 [INFO] processor.data_processor: Conexão com o banco encerrada pelo DataEngine.
2026-02-10 18:06:26 [INFO] processor.data_processor: Conexão com o banco de dados estabelecida pelo DataEngine.
2026-02-10 18:06:26 [INFO] processor.data_processor: EconomicProcessor inicializado e pronto para processamento.
2026-02-10 18:06:26 [INFO] API_ECONOMICA: Motor de Processamento (EconomicProcessor) inicializado com sucesso.
2026-02-10 18:08:38 [INFO] API_ECONOMICA: Acesso ao endpoint de Healthcheck.
2026-02-10 18:08:52 [INFO] API_ECONOMICA: Requisição: Análise global para o país ID 1.
2026-02-10 18:08:52 [INFO] processor.data_processor: Iniciando análise completa para SKU: TODOS | País: 1
2026-02-10 18:08:52 [INFO] processor.data_processor: Buscando preços: SKU=None, Country=1
2026-02-10 18:08:52 [INFO] processor.data_processor: Processando salários para o país ID: 1
2026-02-10 18:08:52 [INFO] processor.data_processor: Processando salários para o país ID: 1
2026-02-10 18:08:52 [INFO] processor.data_processor: Análise finalizada com 2 registros processados.
2026-02-10 18:09:09 [INFO] API_ECONOMICA: Requisição: Ranking global para o SKU HAVAIANAS_TOP.
2026-02-10 18:09:09 [INFO] processor.data_processor: Iniciando análise completa para SKU: HAVAIANAS_TOP | País: TODOS
2026-02-10 18:09:09 [INFO] processor.data_processor: Buscando preços: SKU=HAVAIANAS_TOP, Country=None
2026-02-10 18:09:09 [INFO] processor.data_processor: Processando salários para o país ID: 1
2026-02-10 18:09:09 [INFO] processor.data_processor: Processando salários para o país ID: 2
2026-02-10 18:09:09 [INFO] processor.data_processor: Processando salários para o país ID: 3
2026-02-10 18:09:09 [INFO] processor.data_processor: Análise finalizada com 3 registros processados.
```

Figura 8. Log de execução da API e processamento de consultas sob demanda

Fonte: Resultados originais da pesquisa

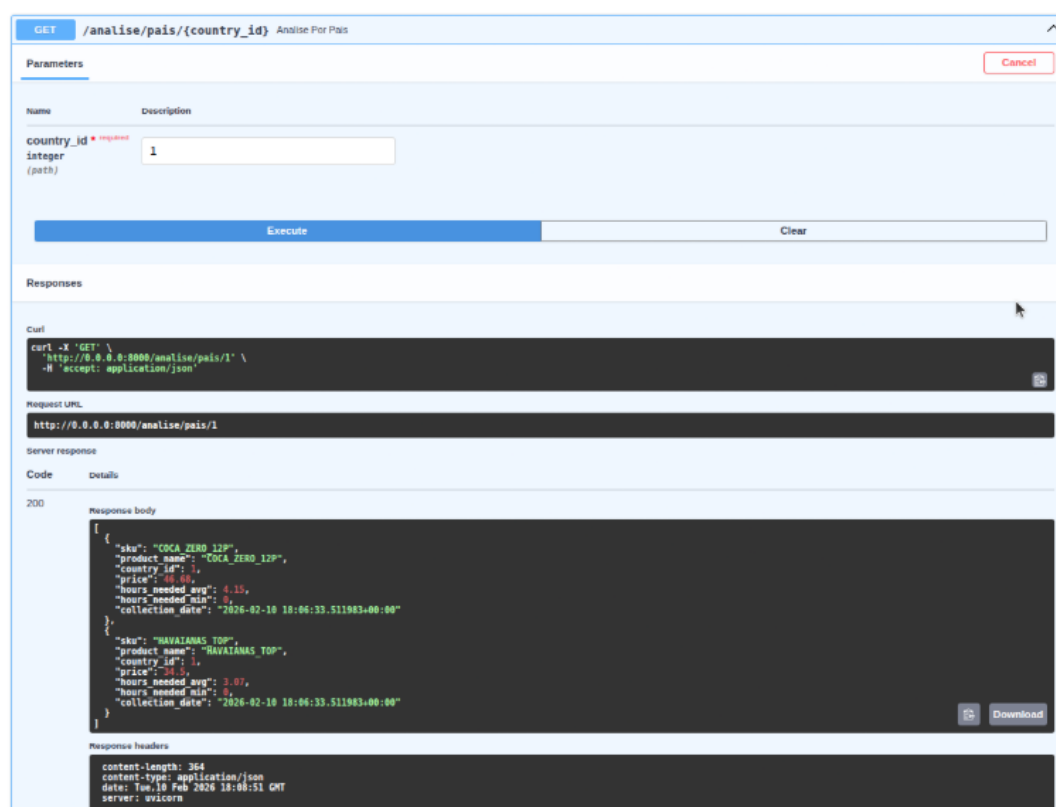


Figura 9. Resposta da API REST para consulta de análise por país

Fonte: Resultados originais da pesquisa

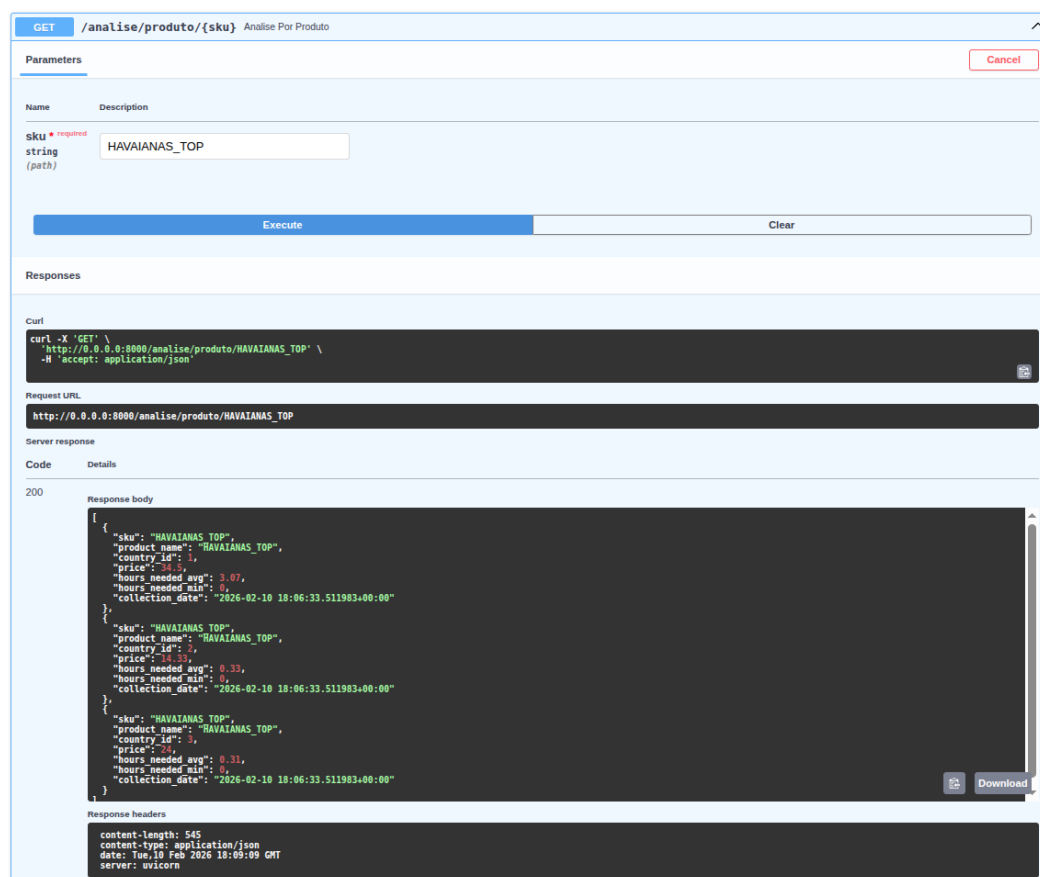


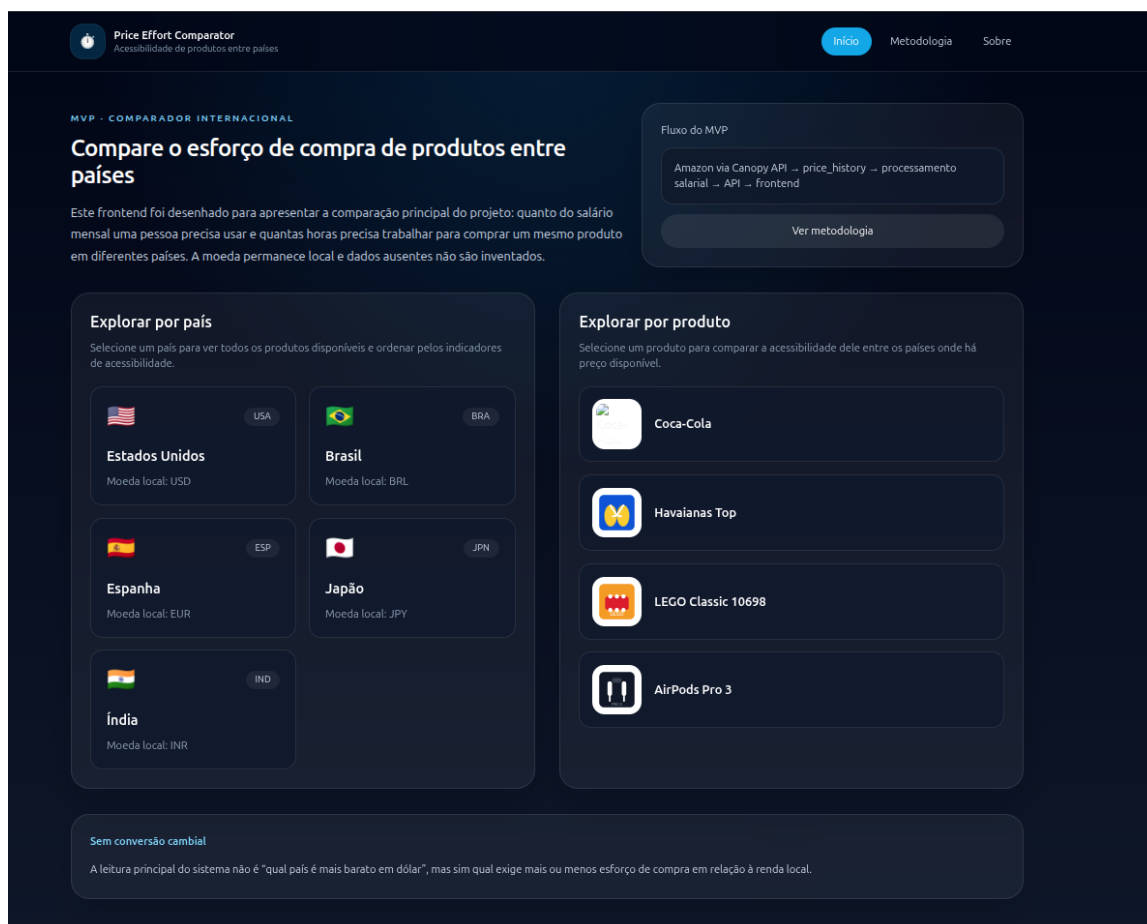
Figura 10. Resposta da API REST para análise comparativa por produto
Fonte: Resultados originais da pesquisa

Implementação e validação do frontend

A camada de apresentação do sistema foi implementada com o objetivo de disponibilizar ao usuário uma interface simples para consulta dos dados processados pelo backend. O frontend foi estruturado para consumir a API desenvolvida e apresentar análises por país e por produto, preservando no backend a responsabilidade pelo processamento das métricas comparativas.

A interface implementada permitiu a navegação entre a página inicial, as telas de análise e as páginas textuais de contextualização do sistema. Os testes realizados indicaram que o frontend foi capaz de consumir corretamente as respostas retornadas pela API e exibir os dados em formato adequado para consulta, o que confirma a viabilidade da camada de apresentação como componente final da solução proposta.

As Figuras 11 e 12 apresentam exemplos da interface web desenvolvida, evidenciando a organização visual das informações e a integração entre a camada de apresentação e os serviços disponibilizados pela API.



Integração entre os componentes do sistema

A integração entre os componentes do sistema constituiu um dos principais resultados observados no desenvolvimento. A solução foi estruturada de modo que cada módulo executasse uma responsabilidade específica dentro do fluxo geral da aplicação, permitindo a separação entre coleta de dados, persistência, processamento, disponibilização das informações e apresentação ao usuário final.

No fluxo implementado, o módulo de coleta realiza a comunicação com as APIs externas e obtém os dados brutos de preços e indicadores econômicos. Em seguida, essas informações são persistidas no banco de dados relacional, preservando o histórico das coletas e assegurando a rastreabilidade dos registros. A API atua como camada intermediária, recuperando os dados armazenados, aplicando as regras de transformação e cálculo das métricas comparativas, e disponibilizando os resultados em formato estruturado para consumo pela interface web.

A camada de frontend, por sua vez, consome os endpoints disponibilizados pela API e apresenta os resultados ao usuário em páginas de consulta por país e por produto. Essa organização evidenciou a separação de responsabilidades entre os componentes, reduzindo o acoplamento entre persistência, processamento e apresentação. Como resultado, a solução mostrou-se adequada ao objetivo de integrar diferentes tecnologias em uma aplicação única, mantendo clareza arquitetural e possibilidade de evolução futura.

Dessa forma, a arquitetura implementada confirmou a viabilidade da proposta do trabalho, demonstrando que a combinação entre coleta automatizada, armazenamento histórico, processamento sob demanda e visualização web pode ser aplicada de forma integrada no contexto do problema estudado.

Análise crítica da solução implementada

A arquitetura modular adotada mostrou-se adequada ao problema proposto, uma vez que permitiu distribuir responsabilidades entre componentes especializados, reduzindo o acoplamento entre coleta, persistência, processamento e apresentação. Essa separação favoreceu a organização do sistema e tornou mais clara a delimitação das responsabilidades técnicas de cada módulo, o que representa uma vantagem importante em soluções que dependem da integração contínua com fontes externas de dados. No contexto da Engenharia de Software, esse resultado é relevante porque demonstra que a adoção de uma estrutura modular contribui não apenas para a clareza arquitetural, mas também para a manutenção e evolução incremental da aplicação.

O processamento sob demanda também se mostrou uma decisão adequada para o escopo do MVP. Em vez de armazenar previamente todas as combinações possíveis de resultados processados, a solução concentrou a persistência nos dados brutos e transferiu para a API a responsabilidade de calcular as métricas no momento da consulta. Essa estratégia reduziu a redundância de armazenamento, preservou a integridade dos dados originais e simplificou a atualização das análises com base na coleta mais recente disponível. Como consequência, a solução conseguiu equilibrar simplicidade estrutural e flexibilidade analítica, o que se mostrou compatível com o objetivo de construir um sistema funcional e evolutivo.

A separação entre backend e frontend também produziu resultados positivos no desenvolvimento da solução. Ao concentrar no backend a lógica de negócio, o processamento

dos indicadores e a integração com o banco de dados, o frontend pôde ser mantido como camada de apresentação e interação, com menor complexidade técnica. Essa decisão favoreceu a clareza da arquitetura, reduziu a sobrecarga da interface web e facilitou a validação do fluxo de dados entre os componentes.

Em conjunto, os resultados indicam que os principais ganhos da solução estiveram na integração consistente entre os módulos, na preservação histórica dos dados coletados e na capacidade de disponibilizar métricas comparativas por meio de uma interface acessível ao usuário. Por outro lado, as limitações observadas, especialmente aquelas relacionadas ao escopo reduzido do MVP e à dependência de fontes externas, mostram que a robustez da arquitetura não elimina as restrições impostas pelo contexto de implementação. Ainda assim, a solução demonstrou aderência ao problema proposto e confirmou que a combinação entre modularidade, processamento sob demanda e separação entre backend e frontend constitui uma abordagem adequada para aplicações desse tipo.

Limitações do MVP e possibilidades de evolução

Embora o sistema desenvolvido tenha demonstrado viabilidade técnica para o escopo proposto, algumas limitações devem ser consideradas. A primeira delas está relacionada ao caráter reduzido do produto mínimo viável, que contemplou um número restrito de países, produtos e indicadores, definido de forma intencional para tornar a implementação compatível com os recursos disponíveis durante o desenvolvimento do trabalho.

Outra limitação relevante refere-se à dependência de fontes externas de dados. No caso dos preços, a utilização da Canopy API impôs restrições operacionais relacionadas ao custo de consumo e ao limite de requisições, o que influenciou diretamente a dimensão do conjunto de produtos e países incluídos no MVP. Além disso, a disponibilidade dos itens pode variar entre os mercados analisados, o que implica ausência legítima de determinados produtos em algumas consultas, sem que isso represente erro de funcionamento do sistema.

Também se destaca como limitação o fato de que, no estágio atual, a comparação foi baseada prioritariamente em indicadores salariais, especialmente salário médio, salário mínimo e métricas derivadas de horas de trabalho necessárias para aquisição dos produtos. Embora essa escolha seja adequada ao objetivo do MVP, ela não esgota as possibilidades analíticas do sistema, que poderá, em versões futuras, incorporar outros indicadores econômicos complementares, como renda disponível, inflação, PIB per capita ou métricas compostas.

Dessa forma, a principal contribuição do trabalho não reside na criação de um novo índice econômico, mas na construção de uma solução de software capaz de integrar fontes heterogêneas e produzir métricas aplicadas de acessibilidade de compra.

A camada de apresentação foi desenvolvida com foco em simplicidade e clareza, priorizando a validação do fluxo de consulta e a integração com a API. Dessa forma, ainda permanecem possibilidades de evolução relacionadas à ampliação da responsividade da interface, inclusão de visualizações gráficas, filtros mais detalhados, ordenações adicionais e mecanismos de comparação histórica.

A preservação dos dados em base histórica constitui, por sua vez, uma das principais oportunidades de evolução da solução. A manutenção contínua dos registros coletados permite que o sistema seja futuramente expandido para análises temporais mais robustas, possibilitando investigar, por exemplo, a evolução conjunta entre preços, salários e esforço de compra ao longo do tempo.

Em conjunto, essas limitações não comprometem a validação da proposta do trabalho, mas delimitam o escopo do MVP e indicam caminhos concretos para evolução da solução em versões futuras.

Conclusão

O desenvolvimento do sistema proposto demonstrou a viabilidade técnica de uma solução web voltada à integração, armazenamento, processamento e disponibilização de dados econômicos provenientes de fontes externas. A aplicação foi capaz de realizar a coleta automatizada de preços e indicadores salariais, preservar os registros em base histórica, processar métricas comparativas sob demanda e disponibilizar os resultados por meio de API e interface web.

Sob a perspectiva da Engenharia de Software, os resultados obtidos indicam que a arquitetura modular adotada atendeu adequadamente ao problema proposto, favorecendo a separação de responsabilidades entre coleta, persistência, processamento e apresentação. A utilização de contêineres, banco de dados relacional, processamento no backend e frontend desacoplado mostrou-se compatível com os objetivos do trabalho, contribuindo para a organização da solução, sua portabilidade e possibilidade de evolução futura.

Do ponto de vista analítico, o sistema permitiu demonstrar que a relação entre preços locais, indicadores salariais e horas de trabalho pode ser utilizada como base prática para consultas comparativas de acessibilidade de compra entre países. Ainda que o escopo do tenha sido deliberadamente reduzido, em razão de limitações operacionais e econômicas da fonte de preços adotada, a solução implementada mostrou-se suficiente para validar a proposta central do trabalho.

Conclui-se, portanto, que o sistema desenvolvido atingiu o objetivo de disponibilizar uma aplicação web capaz de integrar dados heterogêneos e transformá-los em métricas aplicadas de comparação internacional, com ênfase na arquitetura de software e na automação do fluxo de dados. Além de atender ao escopo proposto, a solução estabelece uma base consistente para evoluções futuras, especialmente no que se refere à ampliação do conjunto de produtos, países, indicadores e análises históricas.

Referências

Canopy API. 2024. Canopy API Documentation. Disponível em: <https://www.canopyapi.co/>. Acesso em: 10 fev. 2026.

Docker Inc. 2024. Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: 10 fev. 2026.

FastAPI. 2024. FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 10 fev. 2026.

International Labour Organization [ILO]. 2024. ILOSTAT – Database of Labour Statistics. Disponível em: <https://ilostat.ilo.org/>. Acesso em: 10 fev. 2026.

McKinney, W. 2023. Pandas Documentation. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 10 fev. 2026.

NumPy Developers. 2024. NumPy Documentation. Disponível em: <https://numpy.org/doc/>. Acesso em: 10 fev. 2026.

PostgreSQL Global Development Group [PGDG]. 2024. PostgreSQL Documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 10 fev. 2026.

Pressman, R.S.; Maxim, B.R. 2016. Engenharia de Software: Uma Abordagem Profissional. 8ed. McGraw-Hill Education, Porto Alegre, RS, Brasil.

Python Software Foundation [PSF]. 2024. Python Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 10 fev. 2026.

Reitz, K. 2023. Requests: HTTP for Humans. Disponível em: <https://requests.readthedocs.io/>. Acesso em: 10 fev. 2026.

Sommerville, I. 2016. Engenharia de Software. 10ed. Pearson Education do Brasil, São Paulo, SP, Brasil.

SQLAlchemy. 2024. SQLAlchemy Documentation. Disponível em: <https://docs.sqlalchemy.org/>. Acesso em: 10 fev. 2026.

Uvicorn. 2024. Uvicorn Documentation. Disponível em: <https://www.uvicorn.org/>. Acesso em: 10 fev. 2026.

React. 2026. React Documentation. Disponível em: <https://react.dev/>. Acesso em: 10 fev. 2026.

React Router. 2026. React Router Official Documentation. Disponível em: <https://reactrouter.com/>. Acesso em: 10 fev. 2026.

Tailwind CSS. 2026. Tailwind CSS Documentation. Disponível em: <https://tailwindcss.com/docs/installation/using-vite>. Acesso em: 10 fev. 2026.

TypeScript. 2026. TypeScript Documentation. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 10 fev. 2026.

Vite. 2026. Vite Documentation. Disponível em: <https://vite.dev/guide/>. Acesso em: 10 fev. 2026.

Vite. 2026. Vite Documentation. Documentação oficial. Acesso em: 10 fev. 2026.

Ashenfelter, O. 2001. Cross-country Comparisons of Wage Rates: The Big Mac Index. Princeton University, Princeton, NJ, USA.

International Labour Organization [ILO]. 2026. Wages and Working Time Statistics (COND database): concepts and definitions. Documentação oficial. Acesso em: 10 fev. 2026.
World Bank. 2026. International Comparison Program (ICP) – Methodology. Documentação oficial. Acesso em: 10 fev. 2026.

Apêndice

A. Registros de execução do sistema

```
#1 [internal] load local bake definitions
#1 reading from stdin 662b done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring Dockerfile: 638B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 ...
#4 [auth] library/python:pull token for registry-1.docker.io
#4 DONE 0.0s

#5 [internal] load metadata for docker.io/library/python:3.11-slim
#5 DONE 1.3s

#6 [internal] load .dockerignore
#6 transferring context: 2B done
#6 DONE 0.0s

#7 [1/6] FROM docker.io/library/python:3.11-slim@sha256:20e5d6e279e6d88eaf0e4387c7e9e9e79d11739e63d43923f82de36d2
#7 DONE 0.0s

#7 [internal] load build context
#7 transferring context: 320B done
#7 DONE 0.0s

#8 [1/6] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED

#9 [2/6] RUN apt-get update && apt-get install -y libpq-dev gcc && rm -rf /var/lib/apt/lists/*
#9 CACHED

#10 [3/6] WORKDIR /app
#10 CACHED

#11 [4/6] COPY requirements.txt .
#11 CACHED

#12 [4/6] COPY . .
#12 CACHED

#13 exporting to image
#13 exporting layers done
#13 writing image sha256:29e43179d47f089d1088c8def97eac331af581284ff4b0e365e2f41 done
#13 naming to docker.io/library/comparacao_precos-data_collector done
#13 DONE 0.0s

#14 resolving provenance for metadata file
#14 DONE 0.0s
```

Figura A.1 – Log de build e criação da imagem do container de coleta de dados

Fonte: Resultados originais da pesquisa

```
postgres_db | server started
postgres_db | CREATE DATABASE
postgres_db |
postgres_db |
postgres_db | /usr/local/bin/docker-entrypoint.sh: sourcing /docker-entrypoint-initdb.d/00_set_passwords.sh
postgres_db | CREATE ROLE
postgres_db | CREATE ROLE
postgres_db | ALTER ROLE
postgres_db | ALTER ROLE
postgres_db |
postgres_db | /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/01_schema.sql
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db | CREATE TABLE
postgres_db |
postgres_db | /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/02_permissions.sql
postgres_db | GRANT
postgres_db | GRANT
postgres_db | GRANT
postgres_db | GRANT
postgres_db | GRANT
postgres_db | GRANT
postgres_db |
postgres_db | /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/03_insert_paises.sql
postgres_db | TRUNCATE TABLE
postgres_db | CREATE TABLE
postgres_db | COPY 251
postgres_db | INSERT 0 250
postgres_db | DROP TABLE
postgres_db |
postgres_db | /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/04_load_translations.sql
postgres_db | CREATE TABLE
postgres_db | COPY 179
postgres_db | INSERT 0 179
postgres_db | DROP TABLE
postgres_db |
```

Figura A.2 – Registros de criação da estrutura do banco de dados

Fonte: Resultados originais da pesquisa

```
postgres=# \i /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb-097_insert_sources.sql
postgres=# \i /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb-098_insert_salary_indicators.sql
postgres=# \i /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb-099_insert_categories.sql
postgres=# \i /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb-100_insert_products.sql
postgres=# \i /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb-101_countries.sql
postgres=# \i /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb-102_country_translate_pt.sql
postgres=# \i /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb-103_init.sql
postgres=#
postgres=# [2024-03-29 11:09:58.362 UTC [48] LOG: received fast shutdown request
postgres=# [2024-03-29 11:09:58.364 UTC [48] LOG: aborting any active transactions
postgres=# [2024-03-29 11:09:58.366 UTC [48] LOG: background worker "logical replication launcher" (422) exited with exit code 1
postgres=# [2024-03-29 11:09:58.366 UTC [48] LOG: shutting down
postgres=# [2024-03-29 11:09:58.400 UTC [49] LOG: checkpoint starting: shutdown complete
postgres=# [2024-03-29 11:09:58.466 UTC [49] LOG: checkpoint complete: write 1889 buffers (9.2%); 0 ms; files(s) added: 0, removed: 0, recycled: 0; written=115 k, sync=0.036 s, total=0.108 s; sync files=376, longest=0.004 s, average=0.002 s; distance=4746 MB, estimate=4746 MB; lsn=0/1955408, redo lsn=0/1955408
postgres=#
postgres=# [2024-03-29 11:09:58.476 UTC [48] LOG: database system is shut down
postgres=# done
postgres=# server stopped
postgres=#
postgres=# PostgreSQL init process complete; ready for start up.
```

Figura A.3 – Inserção de dados iniciais e finalização da inicialização do banco de dados

Fonte: Resultados originais da pesquisa

B. Dicionário de dados do banco de dados

Tabela B.1 - Dicionário de Dados: Tabela countries

Campo	Tipo	Chave	Nulo	Descrição
id_country	SERIAL	PK	Não	Identificador interno do país
common_name	VARCHAR(100)	UQ	Não	Nome comum do país (padrão em inglês)
native_name	VARCHAR(100)		Sim	Nome nativo (opcional)
continent	VARCHAR(50)		Não	Continente
iso_2_code	CHAR(2)	UQ	Não	Código ISO-3166-1 alpha-2
iso_3_code	CHAR(3)	UQ	Não	Código ISO-3166-1 alpha-3
base_currency	CHAR(3)		Não	Moeda base (ISO-4217), ex.: BRL, USD

Fonte: Resultados originais da pesquisa

Tabela B.2 - Dicionário de Dados: country_translations

Campo	Tipo	Chave	Nulo	Descrição
id_translation	SERIAL	PK	Não	Identificador da tradução
id_country	INTEGER	FK	Não	País traduzido (countries.id_country)
language_code	CHAR(5)	UQ*	Não	Código do idioma
translated_name	VARCHAR(255)		Não	Nome traduzido no idioma informado

Fonte: Resultados originais da pesquisa

Tabela B.3 - Dicionário de Dados: categories

Campo	Tipo	Chave	Nulo	Descrição
id_category	SERIAL	PK	Não	Identificador da categoria
category_name	VARCHAR(100)	UQ	Não	Nome da categoria

Fonte: Resultados originais da pesquisa

Tabela B.4 - Dicionário de Dados: products

Campo	Tipo	Chave	Nulo	Descrição
sku	VARCHAR(255)	PK	Não	Identificador do produto no sistema
product_name	VARCHAR(255)		Não	Nome do produto
search_term	VARCHAR(255)		Sim	Termo de busca (caso aplicável)
description	TEXT		Sim	Descrição/observações
id_category	INTEGER	FK	Não	Categoria (categories.id_category)

Fonte: Resultados originais da pesquisa

Tabela B.5 - Dicionário de Dados: sources

Campo	Tipo	Chave	Nulo	Descrição
id_source	SERIAL	PK	Não	Identificador da fonte
source_name	VARCHAR(100)	UQ	Não	Nome da fonte

Fonte: Resultados originais da pesquisa

Tabela B.6 - Dicionário de Dados: price_history

Campo	Tipo	Chave	Nulo	Descrição
id_history	BIGINT IDENTITY	PK	Não	Identificador do registro
sku	VARCHAR(255)	FK	Não	Produto (products.sku)
id_source	INTEGER	FK	Não	Fonte (sources.id_source)
id_country	INTEGER	FK	Não	País (countries.id_country)
price	NUMERIC(15,2)		Não	Preço coletado
currency	CHAR(3)		Não	Moeda do preço (ISO-4217)
collection_timestamp	TIMESTAMPTZ	UQ*	Sim	Data/hora da coleta

Fonte: Resultados originais da pesquisa

Tabela B.7 - Dicionário de Dados: labor_indicators

Campo	Tipo	Chave	Nulo	Descrição
id_indicator	SERIAL	PK	Não	Identificador do indicador
indicator_code	VARCHAR(50)	UQ	Não	Código do indicador (chave lógica)
description	VARCHAR(255)		Sim	Descrição do indicador
unit	VARCHAR(20)		Sim	Unidade (ex.: monthly, hourly)
id_source	INTEGER	FK	Sim	Fonte do indicador (ex.: ILOSTAT)

Fonte: Resultados originais da pesquisa

Tabela B.8 - Dicionário de Dados: labor_indicators_history

Campo	Tipo	Chave	Nulo	Descrição
id_salary	BIGINT IDENTITY	PK	Não	
id_country	INTEGER	FK	Não	Identificador do registro País
id_indicator	INTEGER	FK	Não	salary_indicators.id_indicator
id_source	INTEGER	FK	Não	Fonte (sources.id_source)
salary_value	NUMERIC(15,2)		Não	Valor do indicador
currency	CHAR(3)		Não	Moeda do valor (ISO-4217)
reference_year	INTEGER	UQ*	Não	Ano de referência
collection_date	TIMESTAMP TZ		Sim	Data/hora de coleta

Fonte: Resultados originais da pesquisa

Tabela B.9 - Dicionário de Dados: product_asins

Campo	Tipo	Chave	Nulo	Descrição
sku	VARCHAR(255)	PK/FK	Não	Produto, PK Composta
id_country	INT	PK/FK	Não	País, PK Composta
search_code	VARCHAR(50)		Não	Código de busca por país (ex.: ASIN)

Fonte: Resultados originais da pesquisa

Tabela B.10 - Dicionário de Dados: country_stats

Campo	Tipo	Chave	Nulo	Descrição
iso_3_code	CHAR(3)		Sim	País (ISO3)
indicator_code	TEXT		Sim	Código do indicador
salary_value	NUMERIC		Sim	Valor bruto
reference_year	INTEGER		Sim	Ano
currency	CHAR(3)		Sim	Moeda

Fonte: Resultados originais da pesquisa